Student: Yunqi Dong
Andrew ID: yunqid

Project 4

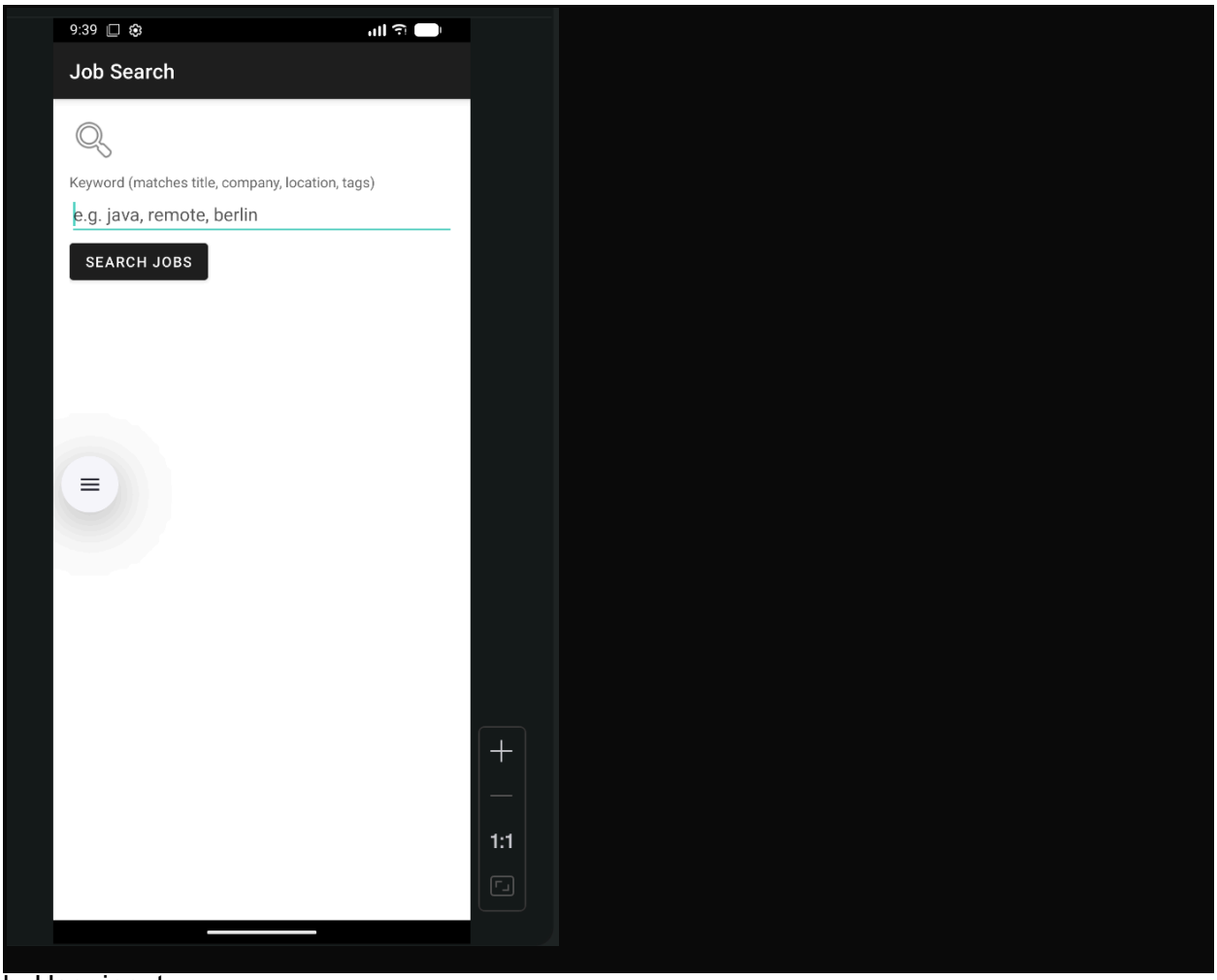
Description:
In Job Search application, Users enter job-related keywords, and the app sends a search request to a Java Servlet deployed in the cloud (via GitHub Codespaces). The server then calls the Arbeitnow public job API, filters the results, and returns a simplified JSON response to the mobile client. The app displays key information such as job title, company, and location on the screen.

Additionally, the server logs each mobile request into MongoDB, and a browser-based dashboard is provided to visualize usage statistics and access logs.

Requirement 1 — Native Android application

a. At least three different kinds of Views**
The main screen layout `android-app/app/src/main/res/layout/activity_main.xml` includes multiple `android.view.View` subclasses, including **ImageView**, **TextView**, **EditText**, **Button**, and **ScrollView** (which contains another **TextView** for results).

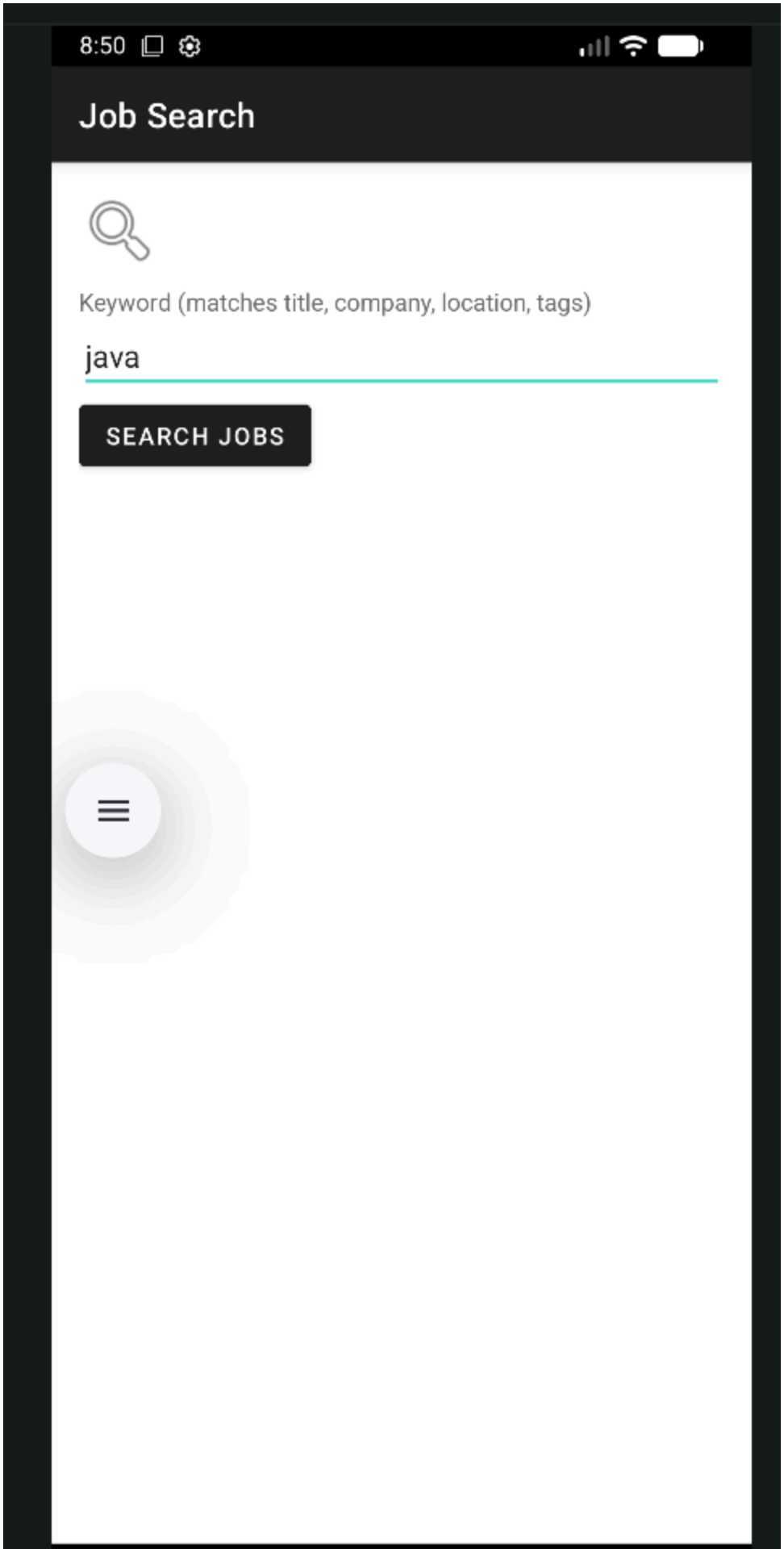
Here is a screenshot of the layout before the job been fetched



b. User input

The user enters a job search **keyword** in `EditText` `keyword_input`.

Here is a screenshot of user searching job using the keyword


8:50
Job Search
Keyword (matches title, company, location, tags)
java
SEARCH JOBS

c. Makes an HTTP request (using an appropriate HTTP method) to your web service

The HTTP request is initiated in `MainActivity.java`. When the user clicks the search button, the method `runSearch()` is executed. In a background thread, the URL is constructed, and the request is sent using `HttpURLConnection` via the method `httpGet(String urlString)`.

The HTTP method used is **GET**. The full request URL has the following format:

{BASE_URL}api/jobs?keyword={keyword}&device={device}

- `BASE_URL` is defined in `build.gradle` as `BuildConfig.BASE_URL`, which corresponds to the public GitHub Codespaces URL (it is recommended to end with `/`).
- `keyword` is the user input from the `EditText` field, encoded using `URLEncoder.encode(..., UTF_8)` before being appended to the query string.
- `device` represents the current device model (`Build.MODEL`). If unavailable, it defaults to `"unknown"`. This value is also URL-encoded and sent for logging purposes on the server.

d. Receives and parses an XML or JSON formatted reply from your web service

The server returns data in **`application/json`** format. In `runSearch()`, the app parses the entire response body using `new JSONObject(body)`.

- If the response contains an `"error"` field, the error message is displayed in the result `TextView`.
- Otherwise, the app reads the `"jobs"` array. For each job entry, it extracts fields such as `title`, `company_name`, `location`, and `url` using `getJSONObject(i)`. These values are then formatted into readable text and displayed to the user.

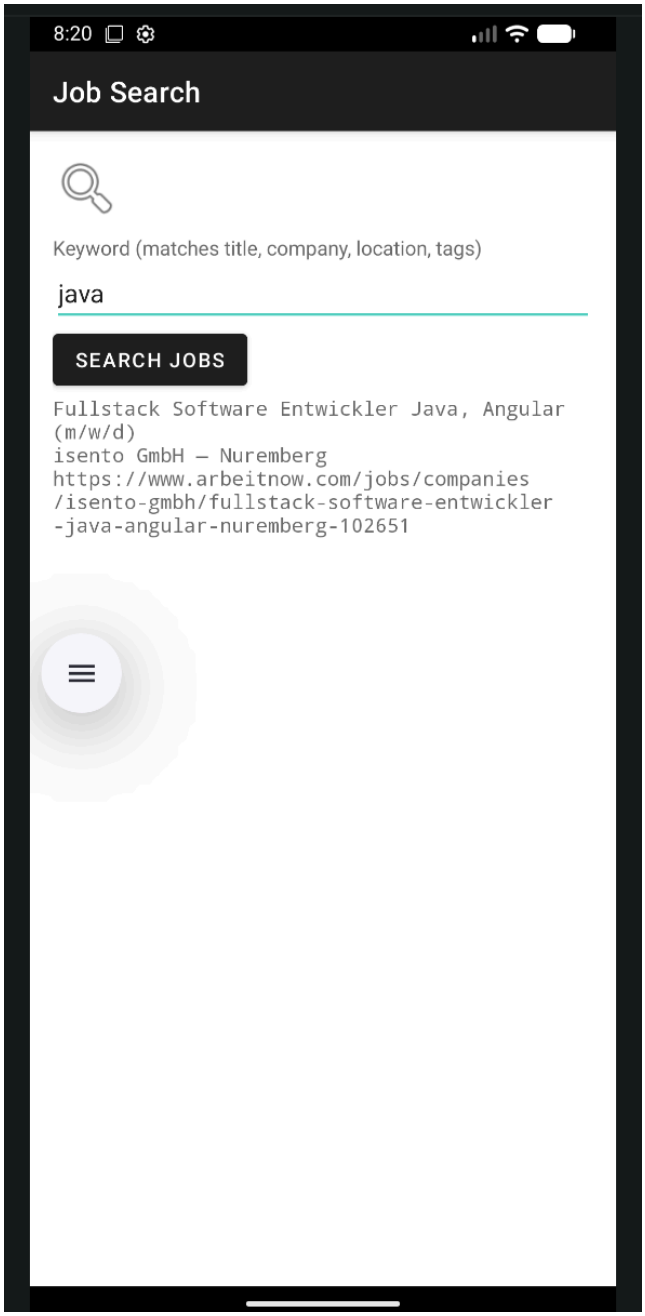
Example of a successful JSON response:

```JSON
{
  "jobs": [
    {
      "title": "Software Engineer",
      "company_name": "Example Corp",
      "location": "Berlin",
      "url": "https://www.arbeitnow.com/view/job/...",
      "remote": true
    }
  ]
}
```

Example of an error JSON response:

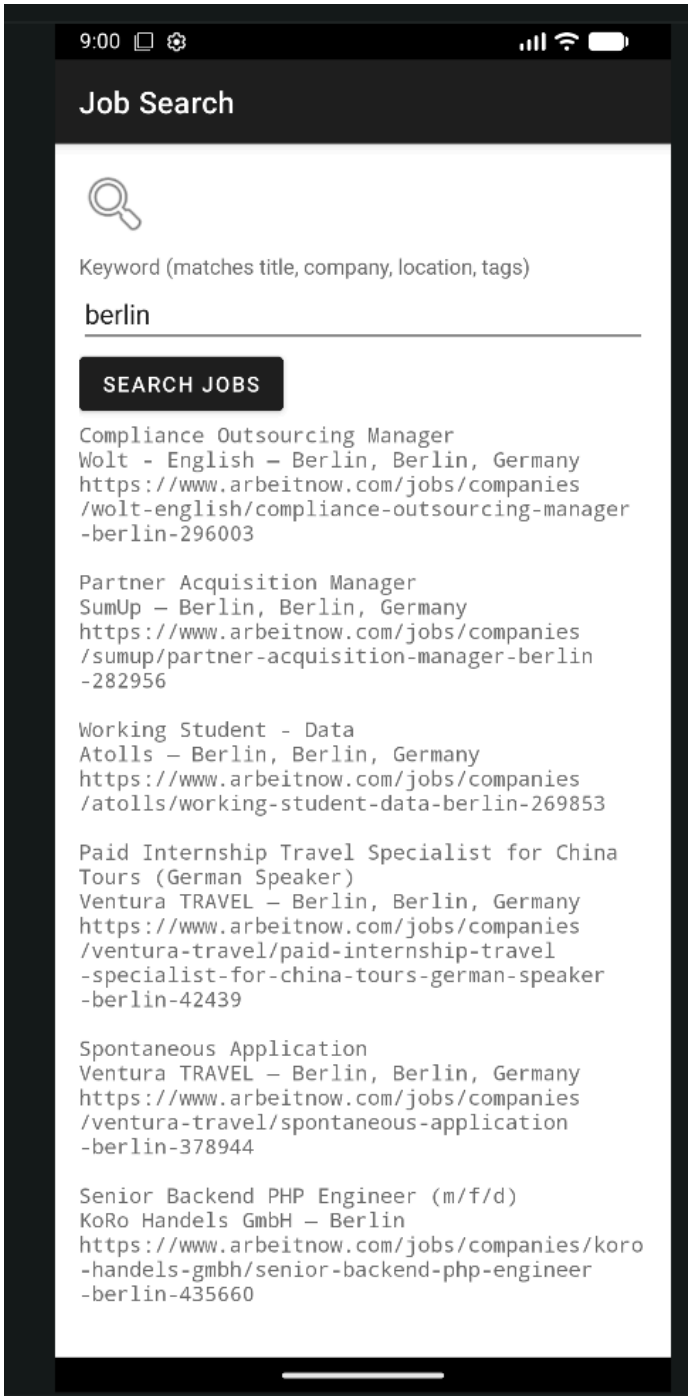
```JSON
{
  "error": "keyword is required"
}
```

e. Display new information
Matching jobs are formatted (title, company, location, URL) and shown in the **results** `TextView`.
Here is the screenshot after the job been returned



f. Repeatable use

The user can run multiple searches from the same activity without restarting the app (each tap on **Search** calls `runSearch()` again).



Requirement 2 — Web service

a. Simple API
A single mobile-facing endpoint is implemented as **`GET /api/jobs`** (`JobSearchServlet`, `@WebServlet(urlPatterns = "/api/jobs")` in `src/main/java/ds/web/JobSearchServlet.java`).

b. Receives HTTP requests from the Android app
The servlet reads query parameters **`keyword`** (required) and **`device`** (optional, for logging), plus the **`User-Agent`** header when present.

c. Business logic and third-party API
Server-side logic fetches **JSON** from the public **Arbeitnow Job Board API** (`https://www.arbeitnow.com/api/job-board-api`) via `ArbeitnowClient`, then filters jobs by keyword using `JobSearchService` (parsing and matching against title, company, location, and tags). This uses a published JSON API, not HTML screen scraping.

d. JSON response to the Android app
Successful responses are **`application/json`** with a **`jobs`** array. Each job object is **slimmed** in `JobSearchService.toSlimJob` to only the fields the mobile client needs (e.g. `title`, `company_name`, `location`, `url`, and `remote` when present), rather than forwarding the full third-party payload. Errors return JSON such as `{"error":"..."}` with an appropriate HTTP status.

Servlets (not JAX-RS)
The API is implemented with **`HttpServlet`** and **`@WebServlet`**, as required.

e. GitHub Classroom / deployment repo]
The repository includes **`Dockerfile`** and **`.devcontainer.json`** at the project root for building and running the web app in **GitHub Codespaces** (see Requirement 7). The Classroom assignment repository should reference or contain these files together with the built **`ROOT.war`** (produced by `mvn package`) or rely on the Docker build stage that runs Maven inside the image (as in this project's `Dockerfile`).

Requirement 4 — Log useful information (≥ 6 fields per mobile interaction)

For each **mobile → `/api/jobs`** request, the service persists one document via `AccessLogRepository.newAccessDocument` / `insertAccessLog` (`src/main/java/ds/web/AccessLogRepository.java`). Each log includes **more than six** attributes, covering:

Logged field	Role

`receivedAt`	When the request was handled
`clientIp`	Client network identity (from `X-Forwarded-For` or remote address)
`userAgent`	HTTP User-Agent from the phone
`deviceModel`	Device model from the `device` query parameter (sent by the app)
`keyword`	Application-specific search parameter
`thirdPartyUrl`	Arbeitnow API URL used
`thirdPartyStatus`	HTTP status from the third-party call
`thirdPartyLatencyMs`	Round-trip time to the third-party API
`jobsReturned`	Number of jobs returned to the phone
`replyHttpStatus`	HTTP status sent back to the mobile client

Dashboard traffic is not logged: only `JobSearchServlet` calls the logging path; `DashboardServlet` only reads MongoDB for the browser UI.

Requirement 5 — Store logs in MongoDB (cloud)

Log documents are stored in MongoDB database **`project4_task2`**, collection **`access_logs`**, using the Java driver (`MongoClient` from `MongoHolder`). Connection uses **`MONGODB_URI`** when set in the deployment environment (e.g. Codespaces / Docker), with a fallback for local development. Logs persist across Tomcat restarts.

Requirement 6 — Operations dashboard (browser)

a. Unique URL
The dashboard is served at **`GET /dashboard`** (`DashboardServlet` → `dashboard.jsp`).

b. At least three operations analytics
`dashboard.jsp` (with data from `AccessLogRepository`) shows:

1. **Total mobile API requests logged** (`countAll`).
2. **Average Arbeitnow API latency (ms)** (`averageThirdPartyLatencyMs`).
3. **Distinct device model values seen** (`distinctDeviceCountEstimate`).

Additionally, a **“Top search keywords”** table lists the most frequent keywords (aggregated in MongoDB), which supports operational insight beyond the three headline metrics.

c. Formatted full logs

A **Recent access logs** section renders the latest entries in an **HTML `<table>`** with columns for time, client IP, device, keyword, third-party HTTP status, third-party latency, jobs returned, and reply HTTP status. Logs are **not** shown as raw JSON or XML.